Experiment 8

Association Mining for Movie Ticket Bookings

This experiment applies the Apriori algorithm to booking baskets in a movie ticket system. It shows how recurring combinations of movie, show time, seat type, payment method, and refreshments can be measured with support, confidence, and lift. The notebook creates a reproducible demonstration dataset and runs in Google Colab.

Platform	Google Colab (Python 3)
Algorithm	Apriori association rule mining
Dataset	120 reproducible synthetic bookings
Thresholds	Minimum support 10%; confidence 40%; lift at least 1.0

Aim

To prepare booking data as transaction baskets, identify frequent itemsets with Apriori, generate positive association rules, and interpret the strongest booking pattern.

Learning outcomes

- Represent each booking as a set of items suitable for association mining.
- Calculate support, confidence, and lift for candidate rules.
- Use discovered patterns to suggest practical show or snack promotions.

Theory

Association rule mining finds items that appear together across transactions. An item can be a categorical value such as Movie=Sky Warriors or Snack=Popcorn. A rule A → B means that transactions containing A also tend to contain B.

Support: Fraction of all bookings that contain both the antecedent and consequent.

Confidence: Fraction of bookings containing the antecedent that also contain the consequent.

Lift: Confidence divided by the support of the consequent. Lift above 1 indicates that the items co-occur more often than expected if they were independent.

Support(A → B) = Support(A ∪ B) | Confidence = Support(A ∪ B) / Support(A) | Lift = Confidence / Support(B)

Dataset and preparation

The notebook generates 120 synthetic bookings with a fixed random seed. Each record contains a booking ID, movie, show slot, seat class, payment method, and two snack indicators. Booking IDs are identifiers and are excluded from the baskets. Movie, show, seat, and payment values become categorical items; popcorn and drink become items only when selected.

The patterns are deliberately generated with varied preferences so the example can demonstrate association mining. They are not observations about a real cinema, real customers, or actual sales.

Procedure

1. Import Python libraries and generate the booking records.
2. Check missing values, duplicate booking IDs, and attribute counts.
3. Convert each booking into a basket and encode item presence as a binary transaction matrix.

4. Run Apriori with minimum support of 10%, pruning candidate itemsets whose subsets are not frequent.
5. Generate rules with confidence of at least 40% and lift of at least 1.0; rank them by confidence, lift, and support.

Colab notebook code and output captures

The following captures show every executed code cell and its printed output. The companion notebook contains the same runnable code.

Cell 1. Import libraries used by the analysis.

Google Colab style cell 1 • Python 3

```
# Cell 1: Import the libraries used in Google Colab
import random
from itertools import combinations
from collections import Counter
import pandas as pd

print("Libraries loaded successfully.")
```

OUTPUT

```
Libraries loaded successfully.
```

Cell 2. Generate the reproducible booking data.

Google Colab style cell 2 • Python 3

```
# Cell 2: Create a reproducible movie booking dataset
random.seed(42)

movies = ["Sky Warriors", "Love in Paris", "Mystery Files", "Galaxy Quest", "Laugh Out Loud"]
shows = ["Morning", "Matinee", "Evening", "Late Night"]
seats = ["Regular", "Premium", "Recliner"]
payments = ["UPI", "Card", "Wallet", "Cash"]

rows = []
for i in range(1, 121):
    # Latent booking preference makes useful co-purchase patterns
    segment = random.choices(["action", "family", "romance", "thriller"], [0.34, 0.27, 0.19, 0.20])[0]
    if segment == "action":
        movie = random.choices(["Sky Warriors", "Galaxy Quest", "Mystery Files"], [0.60, 0.32,
0.08])[0]
        show = random.choices(shows, [0.05, 0.10, 0.65, 0.20])[0]
        seat = random.choices(seats, [0.25, 0.55, 0.20])[0]
    elif segment == "family":
        movie = random.choices(["Laugh Out Loud", "Galaxy Quest", "Sky Warriors"], [0.55, 0.35,
0.10])[0]
        show = random.choices(shows, [0.10, 0.48, 0.35, 0.07])[0]
        seat = random.choices(seats, [0.55, 0.35, 0.10])[0]
    elif segment == "romance":
        movie = random.choices(["Love in Paris", "Laugh Out Loud", "Mystery Files"], [0.70, 0.20,
0.10])[0]
        show = random.choices(shows, [0.05, 0.12, 0.55, 0.28])[0]
        seat = random.choices(seats, [0.30, 0.45, 0.25])[0]
    else:
        movie = random.choices(["Mystery Files", "Sky Warriors", "Love in Paris"], [0.65, 0.25,
0.10])[0]
        show = random.choices(shows, [0.04, 0.10, 0.52, 0.34])[0]
        seat = random.choices(seats, [0.30, 0.50, 0.20])[0]
    payment = random.choices(payments, [0.48, 0.25, 0.20, 0.07])[0]
    popcorn = random.random() < (0.78 if segment in ["action", "family"] else 0.49)
    drink = random.random() < (0.84 if popcorn else 0.35)
    rows.append({"Booking_ID": f"B{i:03d}", "Movie": movie, "Show": show,
                 "Seat": seat, "Payment": payment, "Popcorn": popcorn, "Drink": drink})

df = pd.DataFrame(rows)
print(f"Created {len(df)} bookings with {df.shape[1]-1} booking attributes.")
print(df.head(8).to_string(index=False))
```

OUTPUT

```
Created 120 bookings with 6 booking attributes.
Booking_ID          Movie       Show     Seat Payment  Popcorn  Drink
      B001  Love in Paris    Evening  Regular  Wallet    False  False
      B002   Sky Warriors    Morning  Regular    Card     True   True
      B003  Love in Paris    Evening  Premium  Wallet     True   True
      B004  Love in Paris    Matinee Recliner     UPI     True   True
      B005  Mystery Files Late Night  Premium    Card    False  False
      B006   Galaxy Quest    Evening  Premium    Card     True   True
      B007   Sky Warriors    Matinee  Regular     UPI     True   True
      B008 Laugh Out Loud    Matinee  Regular    Cash     True   True
```

Cell 3. Validate records and summarize booking attributes.

Google Colab style cell 3 • Python 3

```
# Cell 3: Check data quality and summarize booking attributes
print("Missing values:", int(df.isna().sum().sum()))
print("Duplicate booking IDs:", int(df["Booking_ID"].duplicated().sum()))
print("\nMovie counts:")
print(df["Movie"].value_counts().to_string())
print("\nSeat counts:")
print(df["Seat"].value_counts().to_string())
```

OUTPUT

```
Missing values: 0
Duplicate booking IDs: 0

Movie counts:
Movie
Sky Warriors      36
Galaxy Quest      27
Laugh Out Loud    25
Mystery Files     20
Love in Paris     12

Seat counts:
Seat
Premium     60
Regular     43
Recliner    17
```

Cell 4. Convert bookings into transaction baskets.

Google Colab style cell 4 • Python 3

```
# Cell 4: Convert each booking to a basket of present items
transactions = []
for _, row in df.iterrows():
    basket = {
        f"Movie={row['Movie']}", f"Show={row['Show']}",
        f"Seat={row['Seat']}", f"Payment={row['Payment']}"
    }
    if row["Popcorn"]:
        basket.add("Snack=Popcorn")
    if row["Drink"]:
        basket.add("Snack=Drink")
    transactions.append(basket)

items = sorted(set().union(*transactions))
transaction_matrix = pd.DataFrame(
    [{item: int(item in basket) for item in items} for basket in transactions]
)
print(f"Transactions: {len(transactions)} | Encoded items: {len(items)}")
print("First 5 transaction baskets:")
for i, basket in enumerate(transactions[:5], start=1):
    print(f"B{i:03d}: " + ", ".join(sorted(basket)))
```

OUTPUT

```
Transactions: 120 | Encoded items: 18
First 5 transaction baskets:
B001: Movie=Love in Paris, Payment=Wallet, Seat=Regular, Show=Evening
B002: Movie=Sky Warriors, Payment=Card, Seat=Regular, Show=Morning, Snack=Drink, Snack=Popcorn
B003: Movie=Love in Paris, Payment=Wallet, Seat=Premium, Show=Evening, Snack=Drink, Snack=Popcorn
B004: Movie=Love in Paris, Payment=UPI, Seat=Recliner, Show=Matinee, Snack=Drink, Snack=Popcorn
B005: Movie=Mystery Files, Payment=Card, Seat=Premium, Show=Late Night
```

Cell 5. Find frequent itemsets with Apriori.

Google Colab style cell 5 • Python 3

```
# Cell 5: Implement Apriori and find frequent itemsets (minimum support = 10%)
min_support = 0.10
n = len(transactions)
counts = Counter(item for basket in transactions for item in basket)
current = {frozenset([item]): count for item, count in counts.items() if count / n >= min_support}
frequent = dict(current)
k = 2
while current:
    candidates = set()
    prev = list(current)
    for a, b in combinations(prev, 2):
        union = a | b
        if len(union) == k and all(frozenset(s) in current for s in combinations(union, k-1)):
            candidates.add(union)
    next_level = {}
    for candidate in candidates:
        count = sum(candidate.issubset(basket) for basket in transactions)
        if count / n >= min_support:
            next_level[candidate] = count
    frequent.update(next_level)
    current = next_level
    k += 1

freq_rows = [{"Itemset": " + ".join(sorted(items)), "Length": len(items),
              "Support": count / n, "Count": count}
             for items, count in frequent.items()]
freq_df = pd.DataFrame(freq_rows).sort_values(["Support", "Length"], ascending=[False, True])
print(f"Frequent itemsets found: {len(freq_df)}")
print(freq_df.head(12).to_string(index=False, formatters={"Support": "{:.3f}".format}))
```

OUTPUT

```
Frequent itemsets found: 102
                                 Itemset  Length Support  Count
                             Snack=Drink       1   0.658     79
                            Snack=Popcorn       1   0.650     78
                             Show=Evening       1   0.533     64
               Snack=Drink + Snack=Popcorn       2   0.525     63
                             Seat=Premium       1   0.500     60
                              Payment=UPI       1   0.500     60
                Show=Evening + Snack=Drink       2   0.408     49
               Payment=UPI + Snack=Popcorn       2   0.375     45
              Show=Evening + Snack=Popcorn       2   0.367     44
                             Seat=Regular       1   0.358     43
                Payment=UPI + Snack=Drink       2   0.350     42
Show=Evening + Snack=Drink + Snack=Popcorn       3   0.325     39
```

Cell 6. Generate and rank association rules.

Google Colab style cell 6 • Python 3

```
# Cell 6: Generate association rules and rank by confidence, then lift
support_by_set = {items: count / n for items, count in frequent.items()}
rules = []
for itemset, joint_support in support_by_set.items():
    if len(itemset) < 2:
        continue
    for size in range(1, len(itemset)):
        for left_tuple in combinations(itemset, size):
            antecedent = frozenset(left_tuple)
            consequent = itemset - antecedent
            confidence = joint_support / support_by_set[antecedent]
            consequent_support = support_by_set[consequent]
            lift = confidence / consequent_support
            if confidence >= 0.40 and lift >= 1.0:
                rules.append({"Antecedent": " + ".join(sorted(antecedent)),
                              "Consequent": " + ".join(sorted(consequent)),
                              "Support": joint_support, "Confidence": confidence, "Lift": lift})

rules_df = pd.DataFrame(rules).sort_values(["Confidence", "Lift", "Support"], ascending=False)
print(f"Rules meeting confidence >= 0.40 and lift >= 1.00: {len(rules_df)}")
print(rules_df.head(10).to_string(index=False, formatters={
    "Support": "{:.3f}".format, "Confidence": "{:.3f}".format, "Lift": "{:.3f}".format}))
```

OUTPUT

```
Rules meeting confidence >= 0.40 and lift >= 1.00: 183
                                  Antecedent    Consequent Support Confidence  Lift
        Payment=UPI + Seat=Premium + Snack=Drink Snack=Popcorn   0.142      0.944 1.453
  Movie=Galaxy Quest + Show=Evening + Snack=Drink Snack=Popcorn   0.108      0.929 1.429
Movie=Galaxy Quest + Seat=Premium + Snack=Popcorn   Snack=Drink   0.100      0.923 1.402
      Seat=Premium + Show=Evening + Snack=Popcorn   Snack=Drink   0.192      0.920 1.397
               Movie=Galaxy Quest + Snack=Drink Snack=Popcorn   0.133      0.889 1.368
              Movie=Galaxy Quest + Show=Evening Snack=Popcorn   0.133      0.889 1.368
                   Show=Evening + Snack=Popcorn   Snack=Drink   0.325      0.886 1.346
                      Payment=UPI + Snack=Drink Snack=Popcorn   0.308      0.881 1.355
Movie=Sky Warriors + Show=Evening + Snack=Popcorn   Snack=Drink   0.117      0.875 1.329
             Movie=Sky Warriors + Snack=Popcorn   Snack=Drink   0.175      0.875 1.329
```

Cell 7. Interpret the strongest rule and state the conclusion.

Google Colab style cell 7 • Python 3

```
# Cell 7: Summarize the strongest rule for business interpretation
if not rules_df.empty:
    top = rules_df.iloc[0]
    print("Top rule:")
    print(f"IF a booking includes {top['Antecedent']}")
    print(f"THEN it also includes {top['Consequent']}")
    print(f"Support={top['Support']:.3f}, Confidence={top['Confidence']:.3f}, Lift={top['Lift']:.3f}")
    print("Interpretation: lift above 1 indicates the items co-occur more often than expected under
independence.")
else:
    print("No rules met the selected thresholds. Lower minimum support or confidence and rerun.")
print("\nConclusion: Apriori identified recurring combinations in the synthetic booking baskets.")
print("These patterns can inform show-time promotions and snack bundles; they are illustrative, not
customer-level findings.")
```

OUTPUT

```
Top rule:
IF a booking includes Payment=UPI + Seat=Premium + Snack=Drink
THEN it also includes Snack=Popcorn
Support=0.142, Confidence=0.944, Lift=1.453
Interpretation: lift above 1 indicates the items co-occur more often than expected under independence.

Conclusion: Apriori identified recurring combinations in the synthetic booking baskets.
These patterns can inform show-time promotions and snack bundles; they are illustrative, not customer-level
findings.
```

Results and observations

Conclusion

==========

GitHub Repository Link:

https://github.com/Sentry2311/DMBI_EXPERIMENTS/tree/main/Experiment_08_Association_Mining_Aprior i_Python

(Contains complete documentation, dataset, and program code for Experiment 8: Association Mining for Movie Ticket Bookings)